

Figure 1: Doing everything at once?

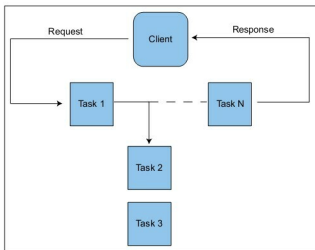


Figure 2: Break the request-response cycle

The general solution

A straightforward solution to the ‘everything in one request-response cycle’ problem is *don’t do everything at once; break the request-response cycle* (see Figure 2). If you can identify which tasks are less important, you can offload these onto another process, and reduce the work that your Web server threads do. By doing this, you reduce the response-to-user time. Please note, what I mean by *less important work* is not work that can be skipped or omitted, but which need not be done immediately, as it does not affect the user response. For example, sending an email is an important part of your application—but it does not directly change the response your server sends to the user. Therefore, whether it’s sent before or after sending the user response does not really matter.

So, your Web server threads take care of receiving the request and doing essential things first—and everything that does not directly affect the user response should be offloaded to another process, so that those things are worked upon in the background, without making the user wait. Also, the reduction in tasks improves the performance of your Web application; the server takes less time to process the request. Thus, your server will also be able to handle a lot more requests.

Queues

A simple way to offload work to other processes is to use queues. Yes, these are the same old queues you studied in your college days. As defined on Wikipedia, a queue is a particular kind of abstract data type or collection in which the entities in the collection are kept in order, and the principal (or only) operations on the collection are the addition of entities to the rear terminal position, and the removal of entities from the front terminal position. Queues are essentially a First-In-First-Out (FIFO) data structure. In our everyday life, queues are used at numerous places to maintain order and efficiency. For example, queues at a ticket counter outside a multiplex are formed to buy tickets on a first-come-first-serve basis, and to also avoid chaos.

As we have seen, you can reduce the load on Web servers by offloading work to other processes. For example, every request that needs to send an email can add a task in the *email queue* at one end; another process (called the *worker process*) can pick up tasks one by one from the other end of the *email queue* and do the required work (in our case, sending an email).

If you’re wondering what queues can help you with, the possibilities are endless. A few general cases where they will prove useful are:

- Data processing that does not directly affect user response.
- Media processing—e.g., conversion/compression of videos, images, etc; SlideShare or Speakerdeck converting your presentation files into their own format, which the application understands.
- Updating caches—this is an important example. You use caches to quickly send responses for requests that have been previously processed, and by some calculations, are not different from the previously processed requests. But after some time, the responses in your caches get stale, if the data used gets changed. You generally wait for a cache miss to happen to regenerate the response and update the cache. Queues can help you with updating caches after certain intervals, so that your cache miss rate reduces.

As a general rule of thumb, you can use queues for offloading off your server just about anything that does not affect the user response. And if you are doing this, then you are working in the direction of improving the overall user experience.

Your first queue

If you understand what a queue is, you can easily implement one within 15 minutes. All you need is a data store (an RDBMS, a key-value store like Redis, or even the main memory) to hold all the tasks (or queue entries) and a script/program written in any programming